Model 1

Code for running Model 1, which calculates and saves the time to extinction across the parameter space (used for Fig. 3).

Background Functions

Initialize chosen TPC and logistic growth function. TPC1 = temperate/generalist, TPC2=tropical/specialist

```
gain[T_] := Exp[-((T - TI)^2) / β];
loss[T_] := ma Exp[mb T] + mc;
r[T_] := (gain[T] - loss[T]) / .755; (*TPC1: temperate*)
(*r[T_]:=(gain[T-2]-loss[T-2])/.4067; (*TPC2: tropical*)*)

TI = 29; (*1: 25, 2: 39*)
β = 180; (*1: 180, 2: 170*)
ma = 0.000005;
mb = 0.4;
mc = 0.05;
```

$$\texttt{nt[t\_, ntlast\_] := } \frac{\texttt{r[Z[[t]]] Exp[r[Z[[t]]]]}}{\left(\frac{\texttt{r[Z[[t]]]}}{\texttt{ntlast}} - \alpha\right) + \alpha\,\texttt{Exp[r[Z[[t]]]]}};$$

Initialize functions for spectral synthesis and spectral mimicry.

```
cmplx[mod_, arg_] := ExpToTrig[mod Exp[I arg]];

SpecSynFourier[γ_, Nobs_, μ_ : 0, σ_ : 1, seed_ : 0] := Module[{
    phase, f, vec},
   If[seed == 0, SeedRandom[], SeedRandom[seed]];
   phase = RandomReal[{0, 2 Pi}, Nobs];
   f = Range[1 / Nobs, 1, 1 / Nobs];
   vec = InverseFourier[
     Join[{0 + 0 I}, Table[cmplx[1 / (f[[i]] ^ (γ / 2)), phase[[i]]], {i, 1, (Nobs - 1)}]],
     FourierParameters → {-1, 1}];
   Standardize[Re[vec]] * σ + μ];

Mimic1[x_, z_] := Module[{y},
   y = x;
   Do [y[[Ordering[z, i][[i]]]] = x[[i]], {i, 1, Length[x]}];
   y];
```

Run Simulation

Runs simulation and stores extinction times. Set parameters for size of run and edit export location. Full sized parameter runs used in final results are shown in parenthesizes. Edit ‘saveto’ for desired location.

```
(*simulation params - variation in σT μT and γ *)
α = .001;
tmax = 100; (*2000*)
maxsims = 5; (*1000*)
γvals = Range[0, 2, .1];(*Autocorrelation: Range[0,2,.1] → 21*)
μTvals = Range[23, 26, 1];(*Mean temp: Range[23,26,.5] → 7*)
σTvals = Range[1, 4, 1];(*St dev of temp: Range[1,4,.5] → 7*)

(*array of temperatures for each parameter set*)
W = ConstantArray[0, {Length[μTvals], Length[σTvals], tmax}];
Do[W[[i, j, k]] = InverseCDF[NormalDistribution[μTvals[[i]], σTvals[[j]]], k * (1 / tmax)],
 {i, 1, Length[μTvals]}, {j, 1, Length[σTvals]}, {k, 1, tmax - 1}]
Do[W[[i, j, tmax]] =
   InverseCDF[NormalDistribution[μTvals[[i]], σTvals[[j]]], (tmax - .5) * (1 / tmax)],
  {i, 1, Length[μTvals]}, {j, 1, Length[σTvals]}];

(*set initial pop size to mean pop size for each parameter set*)
ninit = ConstantArray[0, {Length[μTvals], Length[σTvals]}];
Do[ninit[[i, k]] = (Mean[Table[r[W[[i, k, j]]], {j, 1, tmax}]]) / α,
```

```
 {k, 1, Length[σTvals]}, {i, 1, Length[μTvals]}]

(*initialize array to store extinction times; =tmax if not extinct*)
extTime =
  ConstantArray[tmax, {Length[γvals], Length[μTvals], Length[σTvals], maxsims}];

SetSharedVariable[extTime];
ParallelDo[γ = γvals[[i]];
  μT = μTvals[[j]];
  σT = σTvals[[m]];

  X = SpecSynFourier[γ, tmax];
  Z = Mimic1[W[[j, m]], X];
  envt = Interpolation[Z, InterpolationOrder → 0];

  p = ninit[[j, m]];
  Do[nextp = nt[k, p];
   If[nextp < ((1 / α) * 10^-6), extTime[[i, j, m, l]] = k; Break[],
    p = nextp], {k, 2, tmax}],

  {l, 1, maxsims}, {m, 1, Length[σTvals]},
  {j, 1, Length[μTvals]}, {i, 1, Length[γvals]}] // Timing

saveto = "/SAVETO/";
Export[saveto <> "extTime_tmax" <> ToString[tmax] <> "_dims" <>
  ToString[Length[γvals]] <> "," <> ToString[Length[μTvals]] <> "," <>
  ToString[Length[σTvals]] <> "," <> ToString[maxsims] <> ".m", extTime, "MX"]
```